

PYTHON PROGRAMMING WORKBOOK

From Zero to Real-World Automation

COURSE OVERVIEW

6 Modules · 12+ Hours of Instruction · Beginner to Intermediate

This workbook takes you from your very first line of Python to building real-world automation tools. Every module includes theory, practical code examples, code-along exercises, take-home tasks, and graded assignments — all designed around real African and global tech use cases.

6

Modules

30+

Code Examples

6

Final Projects

Student Name: _____

Start Date: _____ End Date: _____

Instructor: Sulaimon Joel

How to Use This Workbook

This workbook is designed to be hands-on and practical. Here is how to get the most out of it:

For Students

- Read each section carefully before writing any code.
- Type out every code example yourself — do not copy-paste. Typing builds muscle memory.
- Complete the Code-Along Exercise before moving to the take-home task.
- Submit all assignments to your instructor for review.
- If stuck, re-read the theory, then ask your instructor.

For Instructors

- Each module is designed for a 2-hour session.
- Spend the first hour on theory and examples, the second on guided coding.
- Assign the Take-Home Task at the end of each session.
- Collect and review assignments before the next module.

Workbook Icons

Icon	Meaning
□ TIP	Shortcuts, best practices, and pro tricks
□ NOTE	Important concepts to remember
□ WARNING	Common mistakes and how to avoid them
□ CODE-ALONG	In-class guided exercise
□ TAKE-HOME	Complete before the next session
□ ASSIGNMENT	Graded work to submit to instructor

□ NOTE

All code examples in this workbook are written for Python 3.8 and above.

Make sure you have Python 3.x installed — type "python --version" in your terminal.

We recommend VS Code as your editor with the Python extension by Microsoft installed.

MODULE

1

Python Foundations

□ Duration: 2 Hours

Learning Objectives

- Understand what Python is and where it is used in the real world
- Set up Python and VS Code on your computer
- Write and execute your first Python program
- Use variables to store and manipulate data
- Work with Python's core data types: int, float, str, bool
- Convert between data types using type casting

1.1 What is Python?

Python is a high-level, general-purpose programming language known for its readable syntax and massive library ecosystem. Created by Guido van Rossum in 1991, it has become one of the world's most popular languages.

Python powers some of the most impactful technologies of our time:

- Web backends (Instagram, Pinterest, Dropbox use Python)
- Data Science & Machine Learning (Pandas, TensorFlow, scikit-learn)
- Automation & Scripting — automate repetitive tasks in seconds
- API development and consumption
- Cybersecurity and ethical hacking tools
- Finance, Fintech & Cryptocurrency applications
- Social media bots and automation (Twitter, TikTok, WhatsApp)

□ **NOTE**

Python is the #1 most in-demand language for jobs globally (Stack Overflow Survey 2024). In Nigeria's growing tech ecosystem, Python skills open doors in Fintech, EdTech, and automation.

By the end of this course you will be able to build scripts, tools, and automation programs.

1.2 Setting Up Your Environment

Step 1 — Install Python

- Visit <https://python.org/downloads> → download Python 3.x
- During installation, CHECK the box: "Add Python to PATH"
- Verify: open terminal → type `python --version` (should show 3.x.x)

Step 2 — Install VS Code

- Download from <https://code.visualstudio.com>
- Open VS Code → Extensions (Ctrl+Shift+X) → search "Python" → install Microsoft's Python extension

Step 3 — Create and Run a File

- Create a folder called `python_course` on your desktop
- In VS Code: File → New File → save as `hello.py`
- Write your code, then press F5 or right-click → Run Python File in Terminal

1.3 Output & Comments

The `print()` function displays information to the console. Comments explain your code to yourself and others — Python ignores them when running.

```
# Example 1.1 — Hello World & print()

# This is a single-line comment

"""
This is a multi-line comment (docstring).
Great for describing what a program does.
"""

# Basic print
print("Hello, World!")
print("Welcome to Python!")

# print() with multiple values (comma-separated)
print("Name:", "Amaka", "City:", "Lagos")

# F-strings — the cleanest way to format output (Python 3.6+)
name = "Amaka"
city = "Lagos"
print(f"My name is {name} and I live in {city}.")

# Arithmetic inside f-strings
price = 617
litres = 5
print(f"Total cost: N{price * litres}")
```

1.4 Variables

A variable is a named container that stores a value in your computer's memory. In Python, you create one with a single assignment statement — no type declaration needed.

```
# Example 1.2 — Variables

# Creating variables
student_name = "Chidi Okeke"
age          = 22
gpa          = 4.5
```

```
is_enrolled = True

# Python naming rules:
# ✓ snake_case is the Python convention
# ✓ starts with a letter or underscore
# ✗ cannot contain spaces
# ✗ cannot start with a number (3name is invalid)
# ✗ cannot use reserved words (if, for, while, class...)

# Multiple assignment on one line
x = y = z = 0

# Swap two values elegantly
a = 100
b = 200
a, b = b, a
print(a, b) # Output: 200 100
```

1.5 Data Types

Every value in Python has a type. Understanding types prevents bugs and helps you choose the right operations. Use `type()` to inspect a value's type at any time.

Type	Keyword	Example Value	Use Case
Integer	int	age = 25	Counting, IDs, Quantities
Float	float	price = 617.50	Money, GPS coords, Measurements
String	str	name = "Ngozi"	Text, Names, Messages, Codes
Boolean	bool	is_open = True	On/off flags, Conditions
None	NoneType	result = None	Absent / missing values

Example 1.3 – Data Types

```
# Checking types with type()
name = "Tunde"
age = 30
salary = 350000.00
married = False

print(type(name)) # <class "str">
print(type(age)) # <class "int">
print(type(salary)) # <class "float">
print(type(married)) # <class "bool">
```

1.6 Type Casting (Type Conversion)

Type casting converts a value from one type to another. This is critical because `input()` always returns a string — you must convert before doing arithmetic.

Example 1.4 – Type Casting

```
# String → Integer
user_input = "500"          # string from input()
price = int(user_input)     # convert to int
print(price * 3)            # 1500 ✓

# Integer → Float
km = 10
print(float(km))            # 10.0

# Float → Integer (TRUNCATES – does not round!)
pi = 3.99
print(int(pi))              # 3    (not 4!)

# Number → String (needed for string concatenation)
balance = 15000
message = "Balance: N" + str(balance)
print(message)              # Balance: N15000

# Truthiness (bool conversion)
print(bool(0))              # False
print(bool(1))              # True
print(bool(""))             # False (empty string)
print(bool("hello"))        # True  (non-empty string)
```

❏ WARNING

You CANNOT concatenate a string and a number directly.

"Age: " + 25 → TypeError: can only concatenate str (not "int") to str

"Age: " + str(25) → Works perfectly.

1.7 Practical Project — Fuel Station Receipt Printer

Now let's put everything together. We will build a script that prints a formatted fuel receipt. This mirrors a real-world point-of-sale system.

Practical — Fuel Station Receipt (fuel_receipt.py)

```
# =====
# QUICKFILL FUEL STATION – Receipt Printer v1.0
# =====

# Station info
station_name = "QuickFill Fuel Station"
address      = "12 Adeola Odeku Street, Victoria Island, Lagos"
attendant    = "Emeka Okafor"

# Transaction data
fuel_type     = "Premium Motor Spirit (PMS)"
price_per_litre = 617.00    # float: Naira per litre
litres        = 20.5        # float
vat_rate      = 0.075       # 7.5% VAT

# Calculations
```

```
subtotal = price_per_litre * litres
vat       = subtotal * vat_rate
total     = subtotal + vat

# Print receipt
print("=" * 48)
print(f"   {station_name}")
print(f"   {address}")
print("=" * 48)
print(f"   Fuel Type       : {fuel_type}")
print(f"   Price/Litre      : N{price_per_litre:.2f}")
print(f"   Litres           : {litres}L")
print("-" * 48)
print(f"   Subtotal         : N{subtotal:,.2f}")
print(f"   VAT (7.5%)       : N{vat:,.2f}")
print(f"   TOTAL            : N{total:,.2f}")
print("=" * 48)
print(f"   Attendant        : {attendant}")
print("   Thank you for your patronage!")
```

□ CODE-ALONG EXERCISE

Create a file called `my_profile.py`

Print a formatted personal profile card using variables.

Include: name, age, city, course enrolled in, monthly data usage (float), and whether you own a laptop (bool).

Use f-strings and print separator lines with `"="` or `"-"`.

Add at least 3 comments explaining what different sections do.

□ TAKE-HOME TASK

Write a script: `fuel_budget.py`

Ask: How much money do you have? (use a variable, not `input()` yet)

Define the current pump price per litre.

Calculate how many litres you can afford.

Also calculate litres if the price increases by 10%.

Print a formatted comparison showing both scenarios.

□ ASSIGNMENT: Nigerian Currency Converter

File: `currency_converter.py`

Define exchange rate variables: `USD_RATE = 1580`, `GBP_RATE = 2010`, `EUR_RATE = 1710`

Define an amount: `naira_amount = 50000`

Calculate the equivalent in USD, GBP, and EUR.

Print a formatted conversion report with all three results.

BONUS: Add a 1.5% international transfer fee and show the amount after the fee.

BONUS: Show the conversion both ways (Naira → USD and USD → Naira).

MODULE

2

User Input, Errors & Control Flow

□ Duration: 2 Hours

Learning Objectives

- Accept and process real-time input from users with `input()`
- Handle runtime errors gracefully using `try / except`
- Make decisions using `if`, `elif`, and `else` statements
- Combine multiple conditions with `and`, `or`, and `not`
- Build interactive, user-driven programs

2.1 User Input with `input()`

The `input()` function pauses your program and waits for the user to type something. It always returns a string — remember to cast it if you need a number.

Example 2.1 — `input()`

```
# Basic input
name = input("Enter your name: ")
print(f"Hello, {name}!")

# Input with type conversion
age_str = input("Enter your age: ")
age = int(age_str) # convert string to int
print(f"In 5 years you will be {age + 5}")

# One-liner conversion (common pattern)
price = float(input("Enter price: "))
qty = int(input("Enter quantity: "))
print(f"Total: N{price * qty:,.2f}")
```

2.2 Handling Errors with `try / except`

Runtime errors crash your program. A `try/except` block lets you catch errors and respond gracefully instead of showing a scary traceback to the user.

Example 2.2 — `try / except`

```
# Basic try/except
try:
    num = int(input("Enter a number: "))
    print(f"You entered: {num}")
except ValueError:
    print("Error: Please enter a valid number, not text!")

# Multiple except clauses
try:
```

```
a = int(input("Numerator: "))
b = int(input("Denominator: "))
result = a / b
print(f"Result: {result:.2f}")
except ValueError:
    print("Error: Only numbers allowed.")
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")

# else runs only if no exception occurred
# finally runs whether or not there was an error
try:
    amount = float(input("Amount to withdraw: N"))
except ValueError:
    print("Invalid amount.")
else:
    print(f"Withdrawing N{amount:,.2f}...")
finally:
    print("Transaction complete.")
```

□ TIP

Always catch specific exceptions before the general Exception.

Catching Exception as e and printing e helps you see what went wrong during development.

2.3 Conditional Logic: if / elif / else

Conditionals allow your program to make decisions. Python uses indentation (4 spaces) to define code blocks — there are no curly braces.

Example 2.3 – if / elif / else

```
# Comparison operators: == != > < >= <=

# Simple if / else
balance = float(input("Enter account balance: N"))

if balance >= 10000:
    print("Account in good standing.")
else:
    print("Balance below minimum – charge applies.")

# if / elif / else – multiple conditions
score = int(input("Enter exam score (0-100): "))

if score >= 70:
    grade = "A"
elif score >= 60:
    grade = "B"
elif score >= 50:
    grade = "C"
elif score >= 45:
```

```
    grade = "D"
else:
    grade = "F"

print(f"Grade: {grade}")
```

2.4 Compound Boolean Expressions

Use and, or, and not to combine multiple conditions into a single expression.

Example 2.4 – Boolean Logic

```
# and – both conditions must be True
age = int(input("Age: "))
has_id = input("Do you have ID? (yes/no): ")

if age >= 18 and has_id == "yes":
    print("Access granted.")
else:
    print("Access denied.")

# or – at least one condition must be True
payment = input("Payment method (cash/card/transfer): ")

if payment == "cash" or payment == "card":
    print("Processing payment...")
else:
    print("Unsupported payment method.")

# not – inverts the condition
is_closed = False
if not is_closed:
    print("Station is open – welcome!")

# Combining all three
temp = float(input("Temperature (°C): "))
is_raining = input("Is it raining? (yes/no): ") == "yes"

if temp > 35 and not is_raining:
    print("Hot and dry – great day for outdoor work.")
elif temp < 15 or is_raining:
    print("Stay inside – not ideal conditions.")
else:
    print("Comfortable conditions.")
```

Operator	Meaning	Example	Result
and	Both must be True	True and False	False
or	At least one True	True or False	True
not	Inverts the value	not True	False
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True

Operator	Meaning	Example	Result
in	Contained in	"a" in "abc"	True

2.5 Practical Project — ATM Simulator

```
# Practical — ATM Simulator (atm.py)

# =====
# SIMPLE ATM SIMULATOR — Module 2 Project
# =====

CORRECT_PIN = "1234"
balance = 150000.00

print("=" * 40)
print("      WELCOME TO PYBANK ATM")
print("=" * 40)

pin = input("Enter your 4-digit PIN: ")

if pin != CORRECT_PIN:
    print("Incorrect PIN. Card retained. Contact your bank.")
else:
    print(f"PIN correct. Balance: N{balance:,.2f}")
    action = input("Choose: [1] Withdraw [2] Check Balance: ")

    if action == "1":
        try:
            amount = float(input("Enter amount to withdraw: N"))
            if amount <= 0:
                print("Amount must be positive.")
            elif amount > balance:
                print("Insufficient funds.")
            elif amount > 50000:
                print("Daily withdrawal limit is N50,000.")
            else:
                balance -= amount
                print(f"Dispensing N{amount:,.2f}.")
                print(f"New balance: N{balance:,.2f}")
        except ValueError:
            print("Invalid amount entered.")
    elif action == "2":
        print(f"Current balance: N{balance:,.2f}")
    else:
        print("Invalid option selected.")

print("Thank you for banking with PyBank!")
```

□ CODE-ALONG EXERCISE

Extend the ATM code to include a deposit option [3].

Add input validation: the amount must be a multiple of 500 (Nigerian ATM standard).

Print a small receipt at the end showing the transaction type, amount, and new balance.

□ TAKE-HOME TASK

File: grade_calculator.py

Accept marks for 5 subjects from the user.

Calculate the average score.

Display the grade (A/B/C/D/F) and a message: "Distinction", "Credit", "Pass", or "Fail".

Use try/except to handle non-numeric input.

□ ASSIGNMENT: Fuel Station Login & Price Lookup

File: station_login.py

Define credentials: ADMIN_USER = "admin", ADMIN_PASS = "fuel2024"

Accept username and password from the user.

If credentials match, show a menu: [1] Check Price [2] Update Price [3] Exit

Store fuel prices in variables and allow the admin to view them.

After 3 incorrect attempts, print "Account locked." and exit.

BONUS: Use a loop (preview) to allow multiple login attempts.

MODULE

3

Loops, Functions, Modules & PIP

□ Duration: 2 Hours

Learning Objectives

- Use for and while loops to repeat code efficiently
- Control loop flow with break and continue
- Write reusable functions with parameters and return values
- Use lambda for short, anonymous functions
- Import and use Python's built-in modules (math, random, datetime)
- Install third-party packages using PIP

3.1 for Loops

A for loop repeats a block of code for each item in a sequence (list, string, range, etc.).

Example 3.1 – for Loops

```
# Loop over a range of numbers
for i in range(5):          # 0, 1, 2, 3, 4
    print(f"Count: {i}")

# range(start, stop, step)
for i in range(1, 11, 2):   # 1, 3, 5, 7, 9
    print(i)

# Loop over a list
fuels = ["PMS", "AGO", "DPK", "LPG"]
for fuel in fuels:
    print(f"Fuel type: {fuel}")

# enumerate() gives index AND value
for index, fuel in enumerate(fuels, start=1):
    print(f"{index}. {fuel}")

# Loop over a string
for char in "PYTHON":
    print(char, end=" ")    # P Y T H O N
```

3.2 while Loops

A while loop keeps running as long as a condition is True. Use it when you don't know in advance how many iterations you need.

Example 3.2 – while Loops

```
# Basic while loop
count = 1
```

```
while count <= 5:
    print(f"Attempt {count}")
    count += 1

# break – exit the loop early
while True:
    answer = input("Type 'quit' to exit: ")
    if answer == "quit":
        break
    print(f"You typed: {answer}")

# continue – skip current iteration, move to next
for num in range(1, 11):
    if num % 2 == 0:
        continue          # skip even numbers
    print(num)             # prints: 1 3 5 7 9

# while loop for input validation (very common pattern!)
while True:
    try:
        age = int(input("Enter age (1-120): "))
        if 1 <= age <= 120:
            break
        print("Age must be between 1 and 120.")
    except ValueError:
        print("Numbers only, please.")
print(f"Valid age entered: {age}")
```

❏ TIP

The while True + break pattern is the cleanest way to validate input.

Always make sure your while loop has a way to eventually become False — or use break — to avoid infinite loops.

3.3 Functions

A function is a named, reusable block of code. Functions make your code DRY (Don't Repeat Yourself), easier to test, and easier to read.

Example 3.3 – Functions

```
# Defining and calling a function
def greet(name):
    print(f"Hello, {name}! Welcome to Python.")

greet("Fatima")    # Call the function
greet("Emeka")

# Function with return value
def calculate_total(price, qty, vat_rate=0.075):
    subtotal = price * qty
    vat      = subtotal * vat_rate
    return subtotal + vat    # send result back to caller
```

```
total = calculate_total(617.0, 20)
print(f"Total: N{total:,.2f}")

# Multiple return values (returns a tuple)
def fuel_breakdown(price, litres):
    subtotal = price * litres
    vat      = subtotal * 0.075
    total    = subtotal + vat
    return subtotal, vat, total

sub, v, tot = fuel_breakdown(617, 30)
print(f"Sub: N{sub:.2f}  VAT: N{v:.2f}  Total: N{tot:.2f}")

# Default parameter values
def connect(host, port=5432, timeout=30):
    print(f"Connecting to {host}:{port} (timeout={timeout}s)")

connect("localhost")          # uses defaults
connect("api.example.com", 443)
```

3.4 Lambda Functions

Lambda creates a small, one-line anonymous function. Use it for simple operations, especially when passing functions as arguments.

Example 3.4 – Lambda Functions

```
# lambda syntax: lambda arguments: expression

# Regular function vs lambda
def square(x):
    return x ** 2

square_lambda = lambda x: x ** 2

print(square(5))          # 25
print(square_lambda(5))   # 25

# Lambdas shine inside sorted(), map(), filter()
products = [
    {"name": "PMS", "price": 617},
    {"name": "AGO", "price": 980},
    {"name": "LPG", "price": 450},
]

# Sort products by price
sorted_products = sorted(products, key=lambda p: p["price"])
for p in sorted_products:
    print(f"{p['name']:5}  N{p['price']}")

# filter() – keep items where condition is True
prices = [200, 500, 1500, 800, 3000, 150]
cheap = list(filter(lambda x: x < 1000, prices))
print(cheap)              # [200, 500, 800, 150]
```

```
# map() - transform every item in a list
with_vat = list(map(lambda x: round(x * 1.075, 2), prices))
print(with_vat)
```

3.5 Modules & the Standard Library

Python ships with hundreds of built-in modules. Import them with the import statement.

Example 3.5 - Built-in Modules

```
import math
import random
from datetime import datetime

# math module
print(math.pi)           # 3.141592653589793
print(math.sqrt(144))     # 12.0
print(math.ceil(4.2))     # 5
print(math.floor(4.9))    # 4

# random module
print(random.randint(1, 100)) # random int 1-100
print(random.choice(["PMS", "AGO"])) # random item

# datetime module
now = datetime.now()
print(now.strftime("%d %B %Y, %H:%M")) # 01 January 2025, 14:30

# os module - interact with the file system
import os
print(os.getcwd())        # current working directory
print(os.listdir("."))    # list files in current folder
```

3.6 Installing Packages with PIP

PIP is Python's package manager. It connects to PyPI (Python Package Index) and installs third-party libraries you can use in your projects.

Terminal - PIP Commands

```
# Run these commands in your terminal (not in a .py file)

# Check PIP version
pip --version

# Install a package
pip install requests      # HTTP library for calling APIs
pip install colorama      # Colored terminal output
pip install rich          # Beautiful terminal formatting

# Install a specific version
pip install requests==2.31.0

# List all installed packages
```

```
pip list

# Show info about a package
pip show requests

# Save your project dependencies to a file
pip freeze > requirements.txt

# Install from a requirements file
pip install -r requirements.txt
```

□ NOTE

You will use `pip install requests` extensively in Module 6 when we call real APIs. Some networks (including mobile data) may be slow for PIP — download on WiFi when possible.

3.7 Practical Project — Number Guessing Game

```
# Practical — Number Guessing Game (guessing_game.py)

# =====
#  NUMBER GUESSING GAME — Module 3 Project
#  =====
import random

def generate_secret(low=1, high=100):
    """Generate a random secret number."""
    return random.randint(low, high)

def get_guess():
    """Prompt the user and return a valid integer guess."""
    while True:
        try:
            return int(input("Your guess (1-100): "))
        except ValueError:
            print(" Numbers only, please!")

def check_guess(secret, guess):
    """Return hint string based on the guess."""
    if guess < secret:
        return "Too LOW! Guess higher."
    elif guess > secret:
        return "Too HIGH! Guess lower."
    else:
        return "CORRECT!"

def play_game():
    secret = generate_secret()
    attempts = 0
    MAX_TRIES = 7

    print("=" * 40)
```

```
print(" PYTHON NUMBER GUESSING GAME")
print(f" Guess the number (1-100) in {MAX_TRIES} tries!")
print("=" * 40)

for attempt in range(1, MAX_TRIES + 1):
    print(f"\nAttempt {attempt}/{MAX_TRIES}")
    guess = get_guess()
    result = check_guess(secret, guess)
    print(f" {result}")

    if guess == secret:
        print(f" You won in {attempt} attempts!")
        return

print(f"\n Out of tries. The number was {secret}.")

play_game()
```

❑ CODE-ALONG EXERCISE

Add a difficulty setting at the start: Easy (10 tries), Medium (7 tries), Hard (5 tries). After the game ends, ask "Play again? (yes/no)" and restart if they say yes. Track the player's win count across multiple rounds.

❑ TAKE-HOME TASK

File: times_table.py

Ask the user which times table they want to see (e.g., 7).

Use a for loop and a function to print the full table from 1 to 12.

Format the output neatly: "7 x 1 = 7"

BONUS: Ask how many rows to print (not just 12).

❑ ASSIGNMENT: Mini Calculator App

File: calculator.py

Write separate functions: add(), subtract(), multiply(), divide(), modulo(), power()

Each function should take two numbers and return the result.

divide() must handle ZeroDivisionError and return None with an error message.

Build a menu loop: show options [1] Add [2] Subtract ... [7] Exit

Ask for two numbers, call the correct function, print the result, loop back.

BONUS: Add a history list that stores the last 5 calculations.

MODULE

4

Data Structures

□ Duration: 2 Hours

Learning Objectives

- Understand the difference between mutable and immutable data structures
- Use Tuples to store fixed, ordered data
- Use Sets to work with unique collections
- Use Lists to manage ordered, changeable sequences of data
- Use Dictionaries to store key-value pairs like records
- Choose the right data structure for the right problem

4.1 Tuples — Immutable Sequences

A tuple is an ordered, immutable (unchangeable) collection. Use tuples for data that should never change — coordinates, RGB colors, database row schemas.

Example 4.1 — Tuples

```
# Creating tuples
coordinates = (6.5244, 3.3792)      # Lagos GPS coords
fuel_types = ("PMS", "AGO", "DPK", "LPG")
single_item = (42,)                # MUST have trailing comma

# Indexing (0-based)
print(fuel_types[0])               # PMS
print(fuel_types[-1])              # LPG (last item)

# Slicing
print(fuel_types[1:3])              # ("AGO", "DPK")

# Unpacking — assign items to variables
lat, lng = coordinates
print(f"Lat: {lat}, Lng: {lng}")

# Useful tuple methods
nums = (3, 1, 4, 1, 5, 9, 2, 6, 5)
print(nums.count(5))               # 2 (how many times 5 appears)
print(nums.index(9))               # 5 (index of first occurrence of 9)
print(len(nums))                   # 9 (number of items)

# Tuples can be used as dictionary keys (lists cannot!)
distance_map = {
    (0, 0): "Origin",
    (6.5244, 3.3792): "Lagos",
}
```

4.2 Sets — Unique Collections

A set stores unique, unordered items. Perfect for removing duplicates, checking membership quickly, and performing set math (union, intersection, difference).

Example 4.2 — Sets

```
# Creating a set
registered = {"Amaka", "Chidi", "Ngozi", "Emeka", "Amaka"}
print(registered)      # "Amaka" appears only once!

# Add / remove items
registered.add("Fatima")
registered.discard("Chidi")    # safe remove (no error if missing)

# Membership check (much faster than searching a list)
print("Ngozi" in registered)  # True

# Set operations
paid = {"Amaka", "Ngozi", "Fatima"}
enrolled = {"Amaka", "Chidi", "Ngozi", "Emeka", "Fatima"}

print(paid & enrolled)      # Intersection: both paid AND enrolled
print(paid | enrolled)     # Union: paid OR enrolled
print(enrolled - paid)     # Difference: enrolled but NOT paid

# Remove duplicates from a list using a set
log = ["PMS", "AGO", "PMS", "LPG", "AGO", "PMS"]
unique_log = list(set(log))
print(unique_log)
```

4.3 Lists — Ordered Mutable Sequences

Lists are Python's most versatile data structure. They are ordered, changeable, and allow duplicates. You will use them constantly.

Example 4.3 — Lists

```
# Creating and accessing lists
sales = [15000, 23000, 8500, 31000, 19000]
print(sales[0])          # 15000
print(sales[-1])         # 19000 (last item)
print(sales[1:4])        # [23000, 8500, 31000]

# Modifying lists
sales.append(27000)       # add to end
sales.insert(0, 12000)    # insert at index 0
sales.remove(8500)        # remove first occurrence
popped = sales.pop()      # remove & return last item
sales[1] = 25000          # update by index

# Useful methods
print(len(sales))         # number of items
print(sum(sales))         # total of all numbers
print(max(sales))         # largest value
print(min(sales))         # smallest value
print(sorted(sales))      # returns sorted copy
sales.sort(reverse=True)  # sort in-place descending
```

```
# List comprehension – powerful one-liner
prices = [617, 980, 450, 1200]
with_vat = [round(p * 1.075, 2) for p in prices]
print(with_vat)

# Filter with comprehension
expensive = [p for p in prices if p > 600]
print(expensive)    # [617, 980, 1200]
```

4.4 Dictionaries — Key-Value Stores

Dictionaries store data as key: value pairs. They are perfect for representing records, API responses, configuration settings, and database-like objects.

Example 4.4 – Dictionaries

```
# Creating a dictionary
product = {
    "name":      "Premium Motor Spirit",
    "code":      "PMS",
    "price":     617.00,
    "unit":      "litre",
    "in_stock":  True,
}

# Accessing values
print(product["name"])           # Premium Motor Spirit
print(product.get("price"))      # 617.0
print(product.get("discount", 0)) # 0 (default if key missing)

# Modifying dictionaries
product["price"] = 650.00        # update
product["supplier"] = "NNPC"    # add new key
del product["unit"]             # remove key

# Iterating
for key, value in product.items():
    print(f" {key}: {value}")

# List of dictionaries – common for records
inventory = [
    {"name": "PMS", "price": 617, "stock_litres": 50000},
    {"name": "AGO", "price": 980, "stock_litres": 30000},
    {"name": "LPG", "price": 450, "stock_litres": 10000},
]

for item in inventory:
    value = item["price"] * item["stock_litres"]
    print(f"{item['name']}: N{value:,.0f} total value")
```

Structure	Ordered	Mutable	Duplicates	Best For
Tuple ()	Yes	No	Yes	Fixed records, coordinates, DB rows

Structure	Ordered	Mutable	Duplicates	Best For
Set { }	No	Yes	No (unique)	Unique items, fast membership tests
List []	Yes	Yes	Yes	General sequences, ordered data
Dict { : }	Yes (3.7+)	Yes	Keys unique	Records, configs, JSON-like data

4.5 Practical Project — Inventory Manager

Practical — Inventory Manager (inventory.py)

```
# =====
# FUEL STATION INVENTORY MANAGER — Module 4
# =====

inventory = {
    "PMS": {"price": 617.0, "stock": 50000, "unit": "litres"},
    "AGO": {"price": 980.0, "stock": 30000, "unit": "litres"},
    "LPG": {"price": 450.0, "stock": 5000, "unit": "kg"},
}

def show_inventory():
    print("\n" + "=" * 55)
    print(f"  {"PRODUCT":<10} {"PRICE":>10} {"STOCK":>12} {"VALUE":>15}")
    print("-" * 55)
    for code, info in inventory.items():
        value = info["price"] * info["stock"]
        print(f"  {code:<10} N{info['price']:>9,.2f} {info['stock']:>8,}
{info['unit']:<8} N{value:>12,.0f}")
    print("=" * 55)

def add_stock(product_code, amount):
    if product_code in inventory:
        inventory[product_code]["stock"] += amount
        print(f"Added {amount:,} units to {product_code}.")
    else:
        print(f"Product {product_code} not found.")

def sell_fuel(product_code, amount):
    if product_code not in inventory:
        print("Product not found.")
        return
    if amount > inventory[product_code]["stock"]:
        print("Insufficient stock!")
        return
    inventory[product_code]["stock"] -= amount
    revenue = inventory[product_code]["price"] * amount
    print(f"Sold {amount} units. Revenue: N{revenue:,.2f}")

# Demo
show_inventory()
sell_fuel("PMS", 200)
add_stock("AGO", 10000)
show_inventory()
```

□ CODE-ALONG EXERCISE

Add a `low_stock_alert()` function that prints a warning for any product with stock below 5000.

Add a `total_value()` function that returns the total monetary value of all stock.

Add a new product to the inventory dictionary from the command line.

□ TAKE-HOME TASK

File: `shopping_cart.py`

Create an empty list: `cart = []`

Allow the user to add items (name + price) as dictionaries to the cart.

Show the cart contents in a formatted table.

Calculate and show the total price.

Allow the user to remove an item by name.

□ ASSIGNMENT: Student Record System

File: `student_records.py`

Store at least 5 student records as a list of dictionaries.

Each record: `{"name": ..., "id": ..., "scores": [list of 5 scores], "email": ...}`

Write functions: `add_student()`, `view_all()`, `get_average(student)`, `find_by_name(name)`

Display an honor roll — students with average score ≥ 70 .

BONUS: Sort and display the full class ranking by average score.

MODULE

5

Object-Oriented Programming & JSON

□ Duration: 2 Hours

Learning Objectives

- Understand the core principles of Object-Oriented Programming (OOP)
- Create classes with attributes and methods
- Use `__init__` and `self` correctly
- Build class inheritance for code reuse
- Serialize and deserialize data with the `json` module
- Read and write JSON files — the universal data exchange format

5.1 Introduction to OOP

Object-Oriented Programming organizes code around objects — bundles of data (attributes) and behavior (methods). Think of a class as a blueprint and an object as the building constructed from that blueprint.

□ NOTE

A Class is the blueprint (e.g., "BankAccount")

An Object/Instance is a real thing created from the blueprint (e.g., Amaka's account)

Attributes are the data the object holds (e.g., balance, owner)

Methods are the functions the object can perform (e.g., `deposit()`, `withdraw()`)

5.2 Defining Classes

Example 5.1 — Classes & Objects

```
# Define a class
class BankAccount:
    """Represents a simple bank account."""

    # __init__ is called automatically when you create an object
    def __init__(self, owner, account_number, initial_balance=0):
        self.owner = owner
        self.account_number = account_number
        self.balance = initial_balance
        self.transactions = [] # empty list - each account has its own

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
            return
        self.balance += amount
        self.transactions.append({"type": "deposit", "amount": amount})
```

```

        print(f"Deposited N{amount:,.2f}. New balance: N{self.balance:,.2f}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
            return
        self.balance -= amount
        self.transactions.append({"type": "withdrawal", "amount": amount})
        print(f"Withdrew N{amount:,.2f}. New balance: N{self.balance:,.2f}")

    def get_statement(self):
        print(f"\nStatement for: {self.owner} (Acc: {self.account_number})")
        for tx in self.transactions:
            print(f"    {tx['type'].capitalize():<15} N{tx['amount']:,.2f}")
        print(f"    Current Balance: N{self.balance:,.2f}")

# Create objects (instances)
amaka_account = BankAccount("Amaka Obi", "ACC-001", 50000)
chidi_account = BankAccount("Chidi Nwosu", "ACC-002")

amaka_account.deposit(25000)
amaka_account.withdraw(10000)
amaka_account.get_statement()

```

5.3 Inheritance

Inheritance lets a child class extend a parent class, reusing its code and adding or overriding behavior.

Example 5.2 – Inheritance

```

# Parent class
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def describe(self):
        return f"{self.year} {self.make} {self.model}"

    def start(self):
        return f"{self.model} engine started."

# Child class – inherits from Vehicle
class FuelTruck(Vehicle):
    def __init__(self, make, model, year, capacity_litres):
        super().__init__(make, model, year) # call parent __init__
        self.capacity = capacity_litres
        self.current_load = 0

    def load(self, litres):
        if litres > self.capacity:
            print(f"Cannot exceed {self.capacity:,}L capacity.")
        else:
            self.current_load = litres
            print(f"Loaded {litres:,}L into {self.model}.")

```

```
def describe(self):          # override parent method
    base = super().describe()
    return f"{base} - Capacity: {self.capacity:},}L"

truck = FuelTruck("MAN", "TGS 18.440", 2022, 45000)
print(truck.describe())      # uses overridden method
truck.load(40000)
print(truck.start())         # inherited from Vehicle
```

5.4 JSON — The Universal Data Format

JSON (JavaScript Object Notation) is the standard format for data exchange between systems — APIs send JSON responses, config files use JSON, and databases export JSON. Python's json module makes it effortless to work with.

Example 5.3 — JSON

```
import json

# Python dict → JSON string (serialization)
product = {
    "name": "PMS",
    "price": 617.0,
    "in_stock": True,
    "tags": ["fuel", "petrol", "petroleum"],
}

json_string = json.dumps(product, indent=4)
print(json_string)

# JSON string → Python dict (deserialization)
api_response = '{"symbol": "BTC", "price": 65000.50, "change": "+2.3%"}'
data = json.loads(api_response)
print(data["symbol"])    # BTC
print(data["price"])     # 65000.5

# Save Python data to a JSON file
inventory = [
    {"code": "PMS", "price": 617, "stock": 50000},
    {"code": "AGO", "price": 980, "stock": 30000},
]

with open("inventory.json", "w") as f:
    json.dump(inventory, f, indent=4)
print("Inventory saved to inventory.json")

# Read a JSON file back into Python
with open("inventory.json", "r") as f:
    loaded = json.load(f)

for item in loaded:
    print(f"{item['code']}: N{item['price']} ({item['stock']:,}L)")
```

5.5 Practical Project — Customer Registry with JSON Persistence

```
# Practical — Customer Registry (customer_registry.py)

# =====
# CUSTOMER REGISTRY — Module 5 Project
# =====
import json
import os

DATA_FILE = "customers.json"

class Customer:
    def __init__(self, name, phone, email, balance=0):
        self.name = name
        self.phone = phone
        self.email = email
        self.balance = balance

    def to_dict(self):
        """Convert object to dictionary for JSON storage."""
        return {"name": self.name, "phone": self.phone,
                "email": self.email, "balance": self.balance}

    @classmethod
    def from_dict(cls, data):
        """Create a Customer from a dictionary."""
        return cls(data["name"], data["phone"],
                   data["email"], data.get("balance", 0))

class CustomerRegistry:
    def __init__(self):
        self.customers = self._load()

    def _load(self):
        if not os.path.exists(DATA_FILE):
            return []
        with open(DATA_FILE) as f:
            return [Customer.from_dict(d) for d in json.load(f)]

    def save(self):
        with open(DATA_FILE, "w") as f:
            json.dump([c.to_dict() for c in self.customers], f, indent=4)
        print("Registry saved.")

    def add(self, customer):
        self.customers.append(customer)
        self.save()
        print(f"Added: {customer.name}")

    def find(self, phone):
        return next((c for c in self.customers if c.phone == phone), None)

    def list_all(self):
        print(f"\n  {"NAME":<20} {"PHONE":<15} {"BALANCE":>12}")
        print("  " + "-" * 50)
```

```
        for c in self.customers:
            print(f" {c.name:<20} {c.phone:<15} N{c.balance:>10,.2f}")

# Demo
registry = CustomerRegistry()
registry.add(Customer("Amaka Obi", "08012345678", "amaka@email.com", 5000))
registry.add(Customer("Chidi Nwosu", "08023456789", "chidi@email.com", 0))
registry.list_all()
```

□ CODE-ALONG EXERCISE

Add a `top_up(phone, amount)` method to `CustomerRegistry` that increases a customer's balance.

Add a `delete(phone)` method that removes a customer from the list and saves.

Print the total combined balance of all customers.

□ TAKE-HOME TASK

File: `vehicle_fleet.py`

Create a `Vehicle` parent class with: `make`, `model`, `year`, `mileage`.

Create a `Truck` subclass that adds: `payload_capacity`.

Create a `Car` subclass that adds: `num_doors`.

Create 3 `Vehicle` objects (mix of `Truck` and `Car`).

Save all vehicle data to `vehicles.json` and load it back.

□ ASSIGNMENT: Product Catalog App

File: `product_catalog.py`

Design a `Product` class with: `id`, `name`, `category`, `price`, `quantity`.

Build a `Catalog` class that manages a list of `Products`.

Implement: `add_product()`, `remove_by_id()`, `search(name)`, `update_price(id, price)`

Save the full catalog to `catalog.json` on every change.

Load from `catalog.json` on startup so data persists between runs.

BONUS: Add a `low_stock_report()` that shows all products with `quantity < 10`.

MODULE

6

File Handling & Real-World Scripting

□ Duration: 2 Hours

Learning Objectives

- Read from and write to files using Python's built-in file functions
- Use the with statement for safe file handling
- Work with CSV files for tabular data
- Install and use the requests library to make HTTP calls
- Consume REST APIs and process JSON responses
- Build a real automation script that fetches live data from the internet

6.1 Reading Files

Python's `open()` function gives you access to files. Always use the with statement — it automatically closes the file even if an error occurs.

Example 6.1 — Reading Files

```
# Reading a file — full content
with open("sales_report.txt", "r") as f:
    content = f.read()
    print(content)

# Reading line by line
with open("log.txt", "r") as f:
    for line in f:
        print(line.strip())    # strip() removes newline characters

# Read all lines into a list
with open("names.txt", "r") as f:
    lines = f.readlines()      # returns ["line1\n", "line2\n", ...]
    names = [l.strip() for l in lines]  # clean up
    print(names)

# Handle missing file gracefully
try:
    with open("data.txt", "r") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found. Starting fresh.")
    data = ""
```

6.2 Writing & Appending Files

Mode	Symbol	Behavior
Read	"r"	Open for reading. Error if file does not exist.

Mode	Symbol	Behavior
Write	"w"	Create or OVERWRITE. All previous content is lost.
Append	"a"	Create if missing. Adds to END of existing content.
Read+Write	"r+"	Read and write. File must already exist.

Example 6.2 – Writing Files

```
# Writing to a file (creates or overwrites)
with open("receipt.txt", "w") as f:
    f.write("QuickFill Fuel Station\n")
    f.write("=" * 30 + "\n")
    f.write(f"PMS x 20L @ N617 = N{617*20:,}\n")

# Appending to an existing file
with open("sales_log.txt", "a") as f:
    from datetime import datetime
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    f.write(f"[{timestamp}] PMS 20L sold @ N617\n")

# Delete a file
import os

if os.path.exists("temp.txt"):
    os.remove("temp.txt")
    print("File deleted.")
else:
    print("File does not exist.")
```

6.3 Working with CSV Files

CSV (Comma-Separated Values) is the most common format for spreadsheet-like data. Python's csv module makes it easy to read and write.

Example 6.3 – CSV Files

```
import csv

# Writing a CSV file
sales_data = [
    ["Date", "Product", "Litres", "Price", "Total"],
    ["2025-01-01", "PMS", 200, 617, 123400],
    ["2025-01-01", "AGO", 150, 980, 147000],
    ["2025-01-02", "PMS", 350, 617, 215950],
]

with open("sales.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(sales_data)
print("sales.csv created.")

# Reading a CSV file
with open("sales.csv", "r") as f:
    reader = csv.DictReader(f)    # each row is a dictionary
```

```
    for row in reader:
        print(f"{row['Date']} {row['Product']} Total:
N{float(row['Total']):,.0f}")

# Calculate total revenue from CSV
with open("sales.csv", "r") as f:
    reader = csv.DictReader(f)
    total_revenue = sum(float(row["Total"]) for row in reader)
print(f"Total Revenue: N{total_revenue:,.0f}")
```

6.4 HTTP Requests — Calling APIs

The requests library lets you make HTTP calls to web APIs — fetching live data, sending data, and automating web interactions. Install it first: `pip install requests`

Example 6.4 — HTTP Requests

```
# First: pip install requests
import requests

# GET request — fetch data from an API
url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)

# Check status code
print(response.status_code)    # 200 = OK, 404 = Not Found

# Parse the JSON response
data = response.json()
print(data["title"])
print(data["body"])

# GET with query parameters
params = {"userId": 1}
response = requests.get("https://jsonplaceholder.typicode.com/posts",
                        params=params)
posts = response.json()
print(f"Found {len(posts)} posts by user 1")

# POST request — send data to an API
new_post = {"title": "Hello API", "body": "My first POST", "userId": 1}
response = requests.post("https://jsonplaceholder.typicode.com/posts",
                        json=new_post)
print(response.status_code)    # 201 = Created
print(response.json())

# Always handle errors
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()    # raises error for 4xx/5xx
    data = response.json()
except requests.exceptions.ConnectionError:
    print("No internet connection.")
except requests.exceptions.Timeout:
    print("Request timed out.")
except requests.exceptions.HTTPError as e:
```

```
print(f"API error: {e}")
```

❏ TIP

Free test APIs: jsonplaceholder.typicode.com, api.coindesk.com, open-meteo.com (no key needed!)

Always set a timeout to prevent your script hanging forever on a slow connection.
`response.json()` automatically parses the JSON body into a Python dict or list.

6.5 Practical Project — Live Crypto Price Tracker

Practical — Crypto Tracker (`crypto_tracker.py`)

```
# =====
# CRYPTO PRICE TRACKER — Module 6 Project
# Uses CoinGecko API (no API key required!)
# =====

import requests
import json
from datetime import datetime

BASE_URL = "https://api.coingecko.com/api/v3"

def get_price(coin_ids, vs_currency="usd"):
    """Fetch current prices for a list of coins."""
    url = f"{BASE_URL}/simple/price"
    params = {"ids": ", ".join(coin_ids), "vs_currencies": vs_currency,
              "include_24hr_change": "true"}
    try:
        r = requests.get(url, params=params, timeout=10)
        r.raise_for_status()
        return r.json()
    except Exception as e:
        print(f"Error fetching prices: {e}")
        return {}

def format_change(change):
    """Format 24h change with color indicator."""
    arrow = "▲" if change > 0 else "▼"
    return f"{arrow} {abs(change):.2f}%"

def display_prices(data):
    now = datetime.now().strftime("%H:%M:%S")
    print(f"\n CRYPTO PRICES ({now})")
    print(" " + "=" * 45)
    print(f" {"COIN":<15} {"PRICE (USD)":>14} {"24H CHANGE":>12}")
    print(" " + "-" * 45)
    for coin, info in data.items():
        price = info.get("usd", 0)
        change = info.get("usd_24h_change", 0)
        chg_str = format_change(change)
        print(f" {coin.upper():<15} ${price:>13,.2f} {chg_str:>12}")

def save_snapshot(data, filename="price_history.json"):
```

```
"""Append current prices to a history file."""
entry = {"timestamp": datetime.now().isoformat(), "prices": data}
history = []
try:
    with open(filename) as f:
        history = json.load(f)
except FileNotFoundError:
    pass
history.append(entry)
with open(filename, "w") as f:
    json.dump(history, f, indent=4)

# Main
COINS = ["bitcoin", "ethereum", "solana", "binancecoin"]
prices = get_price(COINS)
if prices:
    display_prices(prices)
    save_snapshot(prices)
    print(" Snapshot saved to price_history.json")
```

❑ CODE-ALONG EXERCISE

Add a price alert: ask the user to set a target price for Bitcoin.

After fetching prices, check if BTC has crossed the target (above or below).

Print an alert message: "ALERT: BTC has crossed your target of \$X!"

BONUS: Use a while loop to check prices every 60 seconds (import time; time.sleep(60)).

❑ TAKE-HOME TASK

File: weather_app.py

Use the Open-Meteo API (no key needed): <https://api.open-meteo.com>

Fetch current temperature, wind speed, and weather code for Lagos.

Coordinates: latitude=6.5244, longitude=3.3792

Print a friendly weather summary.

Save the result to weather_log.txt with the current timestamp.

❑ ASSIGNMENT: Automated Sales Report Generator

File: sales_report.py

Read raw sales data from a CSV file (create the CSV file with at least 10 records).

Each row: date, product, litres, price_per_litre

Calculate: total revenue per product, overall total, busiest day, most sold product.

Write a formatted summary report to report.txt

BONUS: Fetch the current exchange rate (USD/NGN) from an API and include the revenue in USD.

Final Projects

Congratulations on completing all 6 modules! It is now time to demonstrate your skills by building a complete real-world application. Choose ONE project from the options below and submit it to your instructor by the deadline.

NOTE

Your project must use concepts from at least 4 different modules.

Submit: your .py source file(s), a README.txt describing how to run the project, and a brief report on what you built.

Projects are evaluated on functionality, code quality, error handling, and presentation.

Project 1 — Fuel Station Management System

Build a complete command-line management system for a petrol station. This is the most comprehensive project and is directly relevant to Nigerian business operations.

Core Features Required

- Inventory management (add, deduct, alert low stock)
- Sales recording with auto-calculated totals
- Daily/weekly revenue report generation
- Customer credit account management
- Pump attendant shift logging
- Save/load all data from JSON or CSV files

Modules & Concepts Used

- Classes & OOP (FuelProduct, SaleRecord, Customer)
- Dictionaries and Lists for records
- File Handling — JSON persistence
- Functions for all operations
- Loops and Menus for navigation
- Error handling for all inputs

Project 2 — Cryptocurrency Price Tracker & Alert Bot

Build an automated price monitoring tool that fetches live crypto prices, sends alerts when targets are hit, and logs price history to disk.

- Fetch live prices from CoinGecko API (free, no key required)
- Let the user set a target high price and target low price for any coin
- Monitor prices in a loop (every 60 seconds using time.sleep)
- Print a colored alert when the price crosses a threshold
- Log all price snapshots with timestamps to a JSON file
- Generate a simple HTML or text report showing the price history
- BONUS: Send an email alert using smtplib when a target is crossed

Project 3 — Social Media Automation Bot

Automate social media interactions using Python. This project introduces industry-standard automation libraries used by marketers and developers worldwide.

Option A — Twitter/X Automation (using Tweepy)

- Install Tweepy: `pip install tweepy`
- Create a Twitter Developer App and get API keys
- Authenticate using your API keys and Bearer Token
- Auto-post a scheduled tweet with today's crypto price or weather update
- Search for tweets with a specific hashtag and like them automatically
- Follow users who tweet about a specific keyword

Option B — Web Automation (using Selenium)

- Install Selenium: `pip install selenium`, and download ChromeDriver
- Automate a browser to open a website and extract information
- Fill in and submit a web form automatically
- Log in to a test website and perform actions
- Extract and save data from a product listing page (scraping)

Project 4 — Text Editor Application (Tkinter GUI)

Build a desktop graphical text editor using Python's built-in Tkinter library — no extra installation required.

- A window with a large text area for typing
- Menu bar with: File (New, Open, Save, Save As, Exit)
- Edit menu: Cut, Copy, Paste, Select All, Find & Replace
- A status bar showing line number, column, and word count
- Font selector allowing the user to change font family and size
- Syntax highlighting BONUS: color Python keywords differently
- Recent files list stored in a `config.json` file

Project 5 — Inventory Management Application

Build a full inventory system suitable for a small business — shop, warehouse, or pharmacy. This is the most practical business application in the list.

- Product management: add, edit, delete, search products
- Each product has: ID, name, category, price, quantity, `reorder_level`
- Sales module: sell items and automatically deduct from stock

- Purchase module: receive new stock and update quantities
- Automatic reorder alert when stock falls below `reorder_level`
- Reports: daily sales, total stock value, best-selling products
- Export reports to CSV file
- All data persisted in JSON or CSV files
- BONUS: Build a simple web interface using Flask

Grading Rubric

Criterion	Points	Description
Functionality	30	All core features work correctly without crashes
Code Quality	20	Clean, well-structured, uses functions and classes
Error Handling	20	All inputs validated, errors caught with try/except
Data Persistence	10	Data saved/loaded from files (JSON or CSV)
Documentation	10	Comments explain code; README explains how to run
Bonus Features	10	Extra functionality beyond the minimum requirements

Appendix — Python Quick Reference

String Methods

Method	Example	Result
<code>upper()</code>	<code>"hello".upper()</code>	<code>"HELLO"</code>
<code>lower()</code>	<code>"HELLO".lower()</code>	<code>"hello"</code>
<code>strip()</code>	<code>" hi ".strip()</code>	<code>"hi"</code>
<code>replace()</code>	<code>"dog".replace("d","l")</code>	<code>"log"</code>
<code>split()</code>	<code>"a,b,c".split(",")</code>	<code>["a", "b", "c"]</code>
<code>join()</code>	<code>",".join(["a","b"])</code>	<code>"a,b"</code>
<code>startswith()</code>	<code>"Python".startswith("P")</code>	<code>True</code>
<code>in operator</code>	<code>"on" in "Python"</code>	<code>True</code>
<code>f-string</code>	<code>f"Hi {name}"</code>	<code>"Hi Amaka"</code>
<code>format</code>	<code>"{: .2f}".format(3.1)</code>	<code>"3.10"</code>

List Methods

Method	Description
append(x)	Add x to end
insert(i, x)	Insert x at index i
remove(x)	Remove first occurrence of x
pop()	Remove and return last item
sort()	Sort in place (ascending)
reverse()	Reverse in place
index(x)	Return index of x
count(x)	Count occurrences of x
len(list)	Number of items
in operator	Check membership

Common Built-in Functions

Function	Purpose	Example
print()	Output to console	<code>print("Hello")</code>
input()	Read user input (returns str)	<code>x = input("Enter: ")</code>
int()	Convert to integer	<code>int("42") → 42</code>
float()	Convert to float	<code>float("3.5") → 3.5</code>
str()	Convert to string	<code>str(100) → "100"</code>
len()	Length of sequence	<code>len([1,2,3]) → 3</code>
range()	Generate number sequence	<code>range(0, 5) → 0,1,2,3,4</code>
type()	Get type of value	<code>type(3.14) → float</code>
sorted()	Return sorted copy	<code>sorted([3,1,2]) → [1,2,3]</code>
sum()	Sum of iterable	<code>sum([10, 20]) → 30</code>
max() / min()	Largest / smallest value	<code>max([3,1,4]) → 4</code>
round()	Round a number	<code>round(3.14159, 2) → 3.14</code>
enumerate()	Index + value from iterable	<code>enumerate(["a", "b"])</code>
zip()	Combine two iterables	<code>zip([1,2], [3,4]) → (1,3), (2,4)</code>